

Программирование и алгоритмизация

(МИСиС, осень-зима 2025)

Полевой Дмитрий Валерьевич
к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

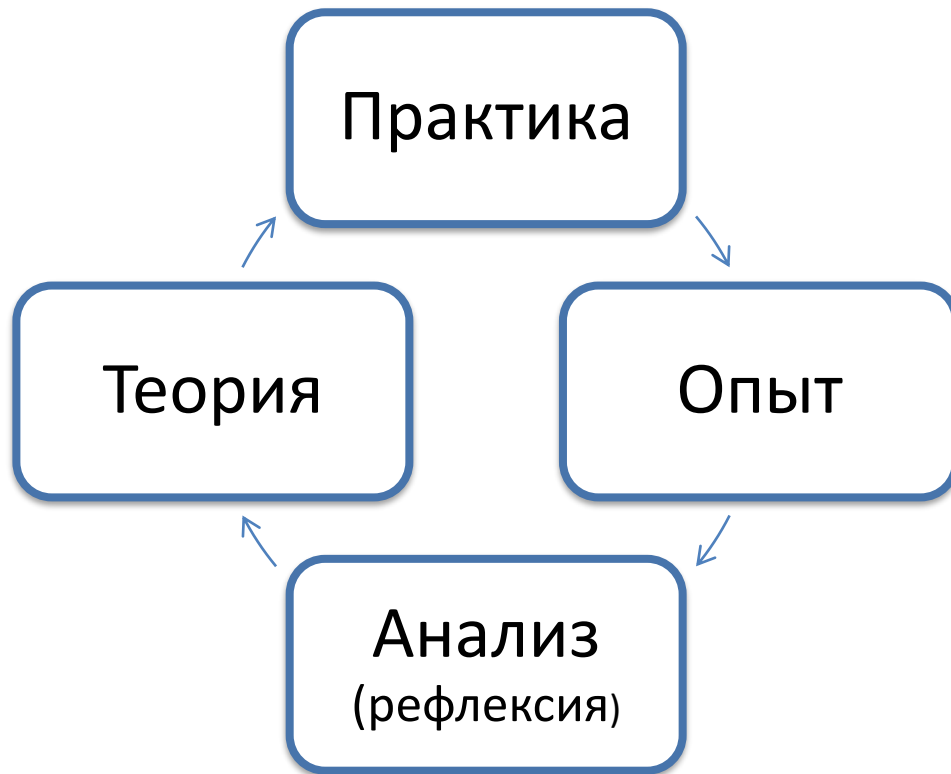
MS Teams - on-line общение

- читайте, следите, спрашивайте
- установите человеческие аватарки
 - ваше лицо
 - хорошо различимо

Стиль и правила кодирования

- обязательны для соблюдения на курсе
- опираются на Google C++ Style Guide
- обсуждаемы
 - совместный документ (Google Docs)
 - аргументы, пояснения, исключения, отклонения

Спираль обучения (цикл Колба)



ваши вопросы

Объектный подход (ОО)

- программа = модель
- все описывается в терминах «объектов»
- объекты могут быть в отношениях
 - часть-целое, являться
- объекты «живут» и «взаимодействуют», т.е. обладают
 - состоянием
 - поведением

Терминология объектного подхода (применительно к ЯП)

- **структура** – пользовательский тип
- **класс** – пользовательский тип ($\exists$ инвариант)
- **экземпляр** – объект типа, переменная
- **поле** – данные, переменная типа/экземпляра
- **метод** – функция типа/экземпляра
- **член** – поле или метод

Доступ к членам

- для экземпляров через оператора доступа к члену «.» (точка)

пример: `center.x = newXPos;`

- для классов через оператор разрешения имени

пример: `Console.ReadLine()`

Метод (экземплярный)

- определен для экземпляра
- функция - вызывается через экземпляр
`obj.method()`
- реализация может обращаться к обычным и статическим членам

пример:

```
int64_t len = arr.ssize(0);
```

Константность (`const`)

- замена препроцессора
- строгий контроль типов
- защита данных
- оптимизация

Константность и защита данных

- инициализация в процессе выполнения
- изменение не предусмотрено
- компилятор пресекает попытки потенциального изменения (проверка присваиваний и модификации)

Константные переменные

- не изменяются после определения

пример:

```
const ptrdiff_t count = vec.size();
```

Константные параметры

- не изменяются внутри функции

пример:

```
int min(const int lhs, const int rhs)
ostream& write(ostream& ostrm, const T& val)
```

ваши вопросы

Перегрузка функций

- использование одного имени для операций с разными наборами аргументов
- одна из форм полиморфизма

пример:

```
int min(const int lhs, const int rhs);
```

```
double min(const double lhs,  
           const double rhs);
```

Форма записи выражений

- традиционная форма

$a + b$

- префиксная форма

$+ a b$

пример:

$a + b$

`operator+(a, b)`

Операторная функция

имя начинается с ключевого слова `operator`,
после которого идет сам оператор

пример:

`operator+`

`operator++`

объявление соответствует грамматике языка
(количество и состав аргументов)

Перегрузка операторов

внутренняя (метод)

`T operator@(const T& rhs)`

внешняя (свободная функция)

`T operator@(const T& lhs, const T& rhs)`

Приоритет операторов

- приоритет операторов определен в стандарте языка
- приоритет операторов не может быть изменен

Унарный оператор

- выражение `@aa` или `aa@`
- нестатический метод без аргументов
`aa.operator@()`
- свободная функция с одним аргументом
`operator@(aa)`

Префиксная и постфиксная формы унарных операторов

префиксный оператор @aa
возвращает результат вычисления

постфиксный оператор aa@
возвращает исходное значение аргумента

Префиксные и постфиксные операторы (сигнатуры)

- префиксный оператор

`T& operator++()`

`T& operator--()`

- постфиксный оператор

`T operator++(int)`

`T operator--(int)`

Бинарный оператор (aa @ bb)

метод с одним аргументом

`aa.operator@ (bb)`

свободная функция с двумя аргументами

`operator@ (aa, bb)`

Ограничения перегрузки операторов

- нельзя создать «новый» оператор
- нельзя изменить число аргументов
- нельзя изменить приоритет

ваши вопросы

Раздельная компиляция

- компилятор (compiler)
 - компиляция независимых компонент
 - генерация объектного кода
- компоновщик (linker)
 - связывание объектного кода
 - генерация исполняемого кода
- преодоление ограничений компилятора
- повторное использование

Этапы сборки программы

- препроцессор
- компилятор → сообщения об ошибках
- компоновщик → сообщения об ошибках

- ВОЗМОЖНА ОПТИМИЗАЦИЯ на этапах
КОМПИЛЯЦИИ И КОМПОНОВКИ
- в IDE происходит «автоматически»

Единица компиляции

исходный файл + препроцессор

→ единица компиляции (трансляции)

- компилируется в собственный объектный файл
- определяет границы видимости

Препроцессор

- директивы включения
`#include`
- директивы условной компиляции
`#if`, `#else`, `#endif`, `#ifndef` и т.д.
- операции со строками
- преобразование аргумента в строку
- конкатенация строк

Макроопределения

- подстановка текстовых значений
`#define, #undef`
- могут определяться на уровне ключей компилятора или опций среды разработки

пример:

```
#define NOMINMAX
```

```
#define min(a,b) ((a) <= (b) ? (a) : (b))
```

Использование препроцессора

МИНИМИЗИРОВАТЬ

- заголовки
 - подключение, стражи включения
- языкозависимые фрагменты кода
- платформозависимые фрагменты кода

Использование библиотек

- ВКЛЮЧИТЬ ЗАГОЛОВОК
- ИСПОЛЬЗОВАТЬ В ИСХОДНОМ КОДЕ
 - объявления
 - определения
 - макросы
- СКОМПОНОВАТЬ БИБЛИОТЕКУ
 - ключи компоновщика или опции проекта в IDE

Реализация модульности в C++

- функция – единица логики исполнения
- модуль (.cpp) – единица организации исходного кода
- общие объявления выносятся в заголовок

Включение заголовочных файлов

- стандартные

```
#include <стандартный_заголовок>
```

- пользовательские

```
#include "пользовательский_заголовок"
```

Порядок поиска

- определяется реализацией компилятора
- может быть причиной “сложных” ошибок
- переменные окружения
- опции компилятора
- текущая папка

Как решать проблемы

- в идеале для каждого .h свой .cpp
- анализировать логику работы `#include`
- анализировать сообщения и смотреть код
- на уровне ключей компиляции или IDE
МОЖНО ВКЛЮЧИТЬ ВЫВОД ПОРЯДКА ВКЛЮЧЕНИЯ

Стандартная библиотека

- новые имена для библиотек Си
`<stdio.h>` → `<cstdio>`
- новые имена для библиотек C++
`<iostream.h>` → `<iostream>`
- не все библиотеки стандартные
`<conio.h>`

ваши вопросы

что такое «массив»?

Массив

- хранилище однотипных элементов
- быстрый доступ к элементам по индексу
- объекты одного типа, расположенные в памяти подряд

АТД «массив»

- создание и уничтожение
- копирование
- доступ к элементам по индексу
- получение размера
- сравнение

АТД «динамический массив»

- удаление элемента (элементов)
- вставка элемента (элементов)
- изменение количества элементов

Массив (встроенный)

избегать использования

- составной пользовательский тип `T [size]`
«массив из `size` элементов типа `T`»
- элементы индексируются от 0 до `size-1`

пример:

```
int divisors[12]{0};
```

Массивы в STL

- заголовки `<array>` и `<vector>`
- массив фиксированного размера
`std::array<T, N>`
- динамический массив
`std::vector<T>`

Массив фиксированного размера

`std::array<T, N>`

- замена встроенным массивам
- произвольный доступ

- объявление `array`

```
template <class T, size_t N>  
struct array;
```

Массив фиксированного размера

`std::array<T, N>`

- замена встроенным массивам
- произвольный доступ

- объявление `array`

```
template <class T, size_t N>  
struct array;
```

Динамический массив

`std::vector<T>`

- замена `new [] / delete []`
- изменяемый размер
- быстрая вставка/удаление в конец
- медленная вставка/удаление в середине и начале

Внутренняя организация

`std::vector`

- автоматическое управление памятью с использованием буферизации
- политика изменения размера буфера
 - выделение по необходимости
 - *2 при добавлении элемента
 - сохранение размера при удалении элемента

Инициализация массивов

- массивы тоже «переменные»
- можно инициализировать списком

пример:

```
struct Mark {  
    std::vector<int> val{2, 3, 4, 5};  
};
```

Доступ к элементам

- по индексу
 - с контролем – `at()`
 - без контроля – `operator[]()`
- с КОНЦОВ
- через итератор (обобщенный вариант)

Доступ к элементам по индексу без контроля выхода за пределы

- `T& operator[] (size_t i)`
- `const T& operator[] (size_t i) const`

пример:

```
data[iBest] = iCurrent;  
iNext = data[j];
```

Доступ к элементам по индексу с контролем выхода за пределы

- `T& at(size_t i)`
- `const T& at(size_t i) const`

пример:

```
data.at(iBest) = iCurrent;  
iNext = data.at(j);
```

Доступ к первому и последнему элементу для пустого контейнера не определен

- `T& front()`
- `const T& front() const`
- `T& back()`
- `const T& back() const`

Размеры `std::array`

- `constexpr size_t size()`
 - число элементов
- `constexpr bool empty()`
 - проверка на пустоту
- `constexpr size_t max_size()`
 - максимальное число элементов (системно)

Размеры `std::vector`

- `size_t size() const`
 - **число элементов**
- `bool empty() const`
 - **проверка на пустоту**
- `size_t max_size() const`
 - **максимальное число элементов (системно)**

Размеры `std::vector`

- `size_t capacity() const`
 - максимальное число элементов (буфер)
- `void reserve(size_t )`
 - гарантировать размер буфера (элементов)
- `void shrink_to_fit()`
 - «поджать» размер буфера

Изменение числа элементов `std::vector`

- `void resize(size_t count)`
- `void resize(size_t count,
 const T& val)`
- «лишнее» отбрасывается
- «новое» заполняется (по умолчанию)

Изменение `std::vector`

- `void push_back(const T& val)`
 - добавление элемента в конец
- `void pop_back()`
 - удаление последнего элемента
- `void clear()`
 - зачистка с сохранением буфера

ваши вопросы

«Волшебные» числа

- литералы (особенно числовые)
- повторяющиеся значения (без имен) усложняют чтение/понимание/модификацию

пример:

```
for (int i(0); i < 256; i += 1) {  
    ...  
}
```

«Волшебные» числа

избегать

- использовать динамические структуры данных
- выносить параметры алгоритмов «наружу»
- использовать локализованные `const` или `constexpr` переменные

пример:

```
const double EPS (1.0E-9) ;
```

Вывод типа из инициализатора (auto)

- тип определяется однозначно (компиляция)
- тип сложен в написании или не известен (шаблон)

пример

```
auto a = 18;
```

```
auto b = 9.0;
```

```
auto x = f(a, b); // ! плохая идея
```

Какие типы можно вывести

- может быть константой
`const auto`
- может быть ссылкой
`auto&`
- может быть константой ссылкой
`const auto&`

Использование auto (рекомендации)

без «фанатизма»

- польза должна превышать вред
- ПОМНИТЬ
 - не уверен — не используй
 - тип есть всегда
 - размер области видимости играет роль

Цикл по диапазону

(range based `for`)

- итерирование элементов последовательности

```
for (range-decl : range-expr)
```

пример

```
for (int v : arr) {  
    // . . .  
}
```

Цикл по диапазону (примеры)

```
std::vector<int> a = {0, 1, 2, 3, 4, 5};  
for (const int& v : a) {  
    std::cout << v << '  ' ;  
}
```

```
for (const auto& v : a) {  
    std::cout << v << '  ' ;  
}
```

ваши вопросы

Обработка текста

- учебные задачи
 - английский алфавит
- реальность
 - национальные кодовые таблицы
 - UNICODE
 - 😊 не в этот раз, но многое будет 😊

Как все начиналось

Си-строка

`<cstring>` - можно посмотреть, как работали

- массив символов `char` заканчивающийся нуль-терминатором
- источник постоянных ошибок

Проблемы манипуляции с Си-строками

- низкоуровневая работа с массивом (указатели)
- без учета константности
- отслеживание конца строки
- отслеживание размера буфера
- большое число функций

Символ

- `char`
- целочисленный тип (не менее 1 байт)
- хранит код символа ASCII (не все печатные)
- `' '` – символьный литерал

пример

```
char ch = 'a';
```

ASCII

US-ASCII – 7 бит

- десятичные цифры
- латинский алфавит
- знаки препинания
- управляющие символы



кодировка – 8 бит

- национальный алфавит (> 127)

dec	oct	hex	ch	dec	oct	hex	ch	dec	oct	hex	ch	dec	oct	hex	ch
0	0	00	NUL (null)	32	40	20	(space)	64	100	40	@	96	140	60	`
1	1	01	SOH (start of header)	33	41	21	!	65	101	41	A	97	141	61	a
2	2	02	STX (start of text)	34	42	22	"	66	102	42	B	98	142	62	b
3	3	03	ETX (end of text)	35	43	23	#	67	103	43	C	99	143	63	c
4	4	04	EOT (end of transmission)	36	44	24	\$	68	104	44	D	100	144	64	d
5	5	05	ENQ (enquiry)	37	45	25	%	69	105	45	E	101	145	65	e
6	6	06	ACK (acknowledge)	38	46	26	&	70	106	46	F	102	146	66	f
7	7	07	BEL (bell)	39	47	27	'	71	107	47	G	103	147	67	g
8	10	08	BS (backspace)	40	50	28	(	72	110	48	H	104	150	68	h
9	11	09	HT (horizontal tab)	41	51	29	)	73	111	49	I	105	151	69	i
10	12	0a	LF (line feed - new line)	42	52	2a	*	74	112	4a	J	106	152	6a	j
11	13	0b	VT (vertical tab)	43	53	2b	+	75	113	4b	K	107	153	6b	k
12	14	0c	FF (form feed - new page)	44	54	2c	,	76	114	4c	L	108	154	6c	l
13	15	0d	CR (carriage return)	45	55	2d	-	77	115	4d	M	109	155	6d	m
14	16	0e	SO (shift out)	46	56	2e	.	78	116	4e	N	110	156	6e	n
15	17	0f	SI (shift in)	47	57	2f	/	79	117	4f	O	111	157	6f	o
16	20	10	DLE (data link escape)	48	60	30	0	80	120	50	P	112	160	70	p
17	21	11	DC1 (device control 1)	49	61	31	1	81	121	51	Q	113	161	71	q
18	22	12	DC2 (device control 2)	50	62	32	2	82	122	52	R	114	162	72	r
19	23	13	DC3 (device control 3)	51	63	33	3	83	123	53	S	115	163	73	s
20	24	14	DC4 (device control 4)	52	64	34	4	84	124	54	T	116	164	74	t
21	25	15	NAK (negative acknowledge)	53	65	35	5	85	125	55	U	117	165	75	u
22	26	16	SYN (synchronous idle)	54	66	36	6	86	126	56	V	118	166	76	v
23	27	17	ETB (end of transmission block)	55	67	37	7	87	127	57	W	119	167	77	w
24	30	18	CAN (cancel)	56	70	38	8	88	130	58	X	120	170	78	x
25	31	19	EM (end of medium)	57	71	39	9	89	131	59	Y	121	171	79	y
26	32	1a	SUB (substitute)	58	72	3a	:	90	132	5a	Z	122	172	7a	z
27	33	1b	ESC (escape)	59	73	3b	;	91	133	5b	[	123	173	7b	{
28	34	1c	FS (file separator)	60	74	3c	<	92	134	5c	\	124	174	7c	
29	35	1d	GS (group separator)	61	75	3d	=	93	135	5d	]	125	175	7d	}
30	36	1e	RS (record separator)	62	76	3e	>	94	136	5e	^	126	176	7e	~
31	37	1f	US (unit separator)	63	77	3f	?	95	137	5f	_	127	177	7f	DEL (delete)

Управляющие последовательности

\'	single quote	\f	form feed - new page
\"	double quote	\n	line feed - new line
\?	question mark	\r	carriage return
\\	backslash	\t	horizontal tab
\a	audible bell	\v	vertical tab
\b	backspace		

**как вывести код
символа?**

Вывод кода символа в поток

вывести значение типа int

```
char ch = ' ';  
std::cout << static_cast<int>(ch);
```

Строковый литерал

`"текст внутри двойных кавычек"`

из каких байтов состоит?

Строковый литерал

- значения символов зависят от кодировки файла исходного кода

ваши вопросы

Строковый класс (`std::string`)

- стандартная библиотека `<string>`
- стандарт не определяет конкретный способ хранения строк в памяти
- стандарт допускает реализацию с подсчетом ссылок
- часто оптимизировано для коротких строк

Шаблонное объявление

```
template <
    class Ch,
    class Tr = char_traits<Ch>,
    class Al = allocator<Ch> >
class basic_string;

typedef basic_string<char> string;
```


Создание строки

- пустая строка

пример:

```
string str;
```

- инициализация строковым литералом

пример:

```
string str("misis");
```

- копирование существующей

пример:

```
string str(strOld);
```

Инициализация строки символом

- невозможна инициализация символом

пример:

```
string('a') // ! ошибка !
```

- возможна инициализация несколькими экземплярами символа

пример:

```
string('a', cnt)
```

Присвоение

- `assign`
- `operator=`

пример:

```
strFI.assign(str);
```

```
strFI.assign("");
```

```
strFI = str;
```

```
strFI = "test text";
```

Размер строк

- размер буфера
`capacity`
- резервирование
`reserve`
- длинна строки
(без нуля)
`length`
`size`
- изменение длины
(дополняется пробелами
или усекается)
`resize`

Преобразование в С-строку

```
const Ch* c_str() const
```

– записывает символы в массив, дополняя их
финальным нулем

```
const Ch* data() const
```

– записывает символы в массив

- копирование из внутреннего буфера во
внешний
copy

Присоединение к строке

- `append`
- `operator+=`

пример:

```
strFI.append(strFamilyName);  
strFI.append(" ");  
strFI += strName;
```

Конкатенация строк

- объединение содержимого строк
`operator+`
- избегать массового использования

пример:

```
strFI = strFamilyName + string(" ")  
      + strName;
```

Вставка символов

- `insert` - вставка подстроки

пример:

```
strFI.insert(posName, name);
```

- `replace` - вставка с заменой

пример:

```
st.replace(pos, lnOld, lnNew, name);
```

— вставка начинается перед указанной позицией

Поиск в строке

- `std::string::size_type`
- `std::string::npos` – при неудачном поиске
- `find`
 - заданная группа символов
- `rfind`
 - с конца, заданная группа символов

Поиск совпадающих

- `find_first_of`
 - первый символ из заданной группы
- `find_last_of`
 - последний символ из заданной группы

Поиск отличающихся

- `find_first_not_of`
 - первый символ не совпадающий ни с одним из символов заданной группы
- `find_last_not_of`
 - последний символ не совпадающий ни с одним из символов заданной группы

Доступ к символу

- `operator[]`
 - по индексу без контроля выхода за границы
- `at`
 - по индексу с контролем выхода за границы
 - при ошибке исключение `std::out_of_range`

Удаление символов

- `erase` – удаление символов
- `clear` – очистка строки

пример:

```
st.clear();
```

```
st.erase(lBeg, string::npos);
```

```
st.erase(lBeg, lEnd - lBeg + 1);
```

Проверка на пустоту

```
bool empty() const
```

пример:

```
if (!str.empty()) {  
    // строка не пустая  
    //обработка  
}
```

ПОТОКОВЫЙ ВВОД И ВЫВОД

- **ВЫВОД**

`operator<<`

- **ВВОД**

`operator>>`

Сравнение символов

- лексикографическое сравнение
 - упорядочение в алфавите
 - упорядочение в кодовой таблице
- символы нижнего регистра меньше символов верхнего

Сравнение строк

- происходит лексикографическим сравнением символов до первого несовпадения
- упорядоченность строк определяется порядком несовпадающих символов

Функции сравнение строк

- `operator==`
- `operator!=`
- `operator>`
- `operator<`
- `operator>=`
- `operator<=`
- `compare`

ваши вопросы

ПОТОКОВЫЙ ВЫВОД И ВВОД

- вывод – преобразование объектов (типа) в последовательность символов
- ввод – запрос последовательности символов и преобразование ее в объект (типа)

Тип потока (базовый класс)

- `ostream` – вывода (выходной)
- `istream` – ввода (входной)
- `iostream` – ввода и вывода
(двунаправленный)

Заголовочные файлы

<iostream>

– стандартные потоки

<fstream>

– файловые потоки

<sstream>

– строковые потоки

<iosfwd>

– опережающие
объявления

<iomanip>

– манипуляторы

<streambuf>

– буферы

Преимущества стандартной библиотеки ввода/вывода

- контроль типов передаваемых данных
- использование одного кода для работы с разными потоками (через полиморфизм)
- расширяемость
 - ВВОД И ВЫВОД НОВЫХ ТИПОВ
 - НОВЫЕ КЛАССЫ ПОТОКОВ

Стандартные потоки

- глобальные переменные
- возможно перенаправление
- консольный ввод и вывод (`cin` и `cout`)
- протоколирование ошибок (`cerr`)
- ведение логов (`clog`)

Базовые операции

- открытие (создание)
- `operator<<`
 - запись в поток (поместить данные)
- `operator>>`
 - чтение из потока (извлечь данные)
- проверка состояния
- закрытие (уничтожение)

Манипулятор

- специальный объект, вызывающий функцию потока
- стандартные манипуляторы определены в пространстве имен `std`

пример:

```
auto& strm = std::cout;  
strm << std::hex << val << std::endl;
```

Буферизация потоков

- для записи и чтения используется буфер, который освобождается при переполнении или по требованию

пример:

```
strm.flush();
```

```
strm << obj << endl;
```

Синтаксис форматированного ВВОДА И ВЫВОДА

перегруженные операторы двоичного сдвига

```
cout << "a=" << a;
```

означает

```
operator<<(cout, "a=") .operator<<(a) ;
```

Как читаются строки

при вводе строки

"Hello, I am computer?"

ВОТ ТАКИМ СПОСОБОМ

```
std::string str;
```

```
std::cin >> str;
```

что будет в переменной str?

Ввод текста из потока

- переменные типа `std::string` зачитываются до разделителя
- разделители
(пробел, табуляции, конец строки и т.д.)

Построчный ввод текста

для чтения текстового файла по строкам
используйте функцию

`getline` определенную в `<string>`

пример:

```
std::string str;  
while (!getline(std::cout, str).fail()) {  
    // ...  
}
```

Неформатированный ввод

- чтение отдельного символа или последовательности из потока
- `get`
 - прочитать из потока отдельный символ
- `read`
 - прочитать из потока последовательность

Неформатированный вывод

- вывод отдельного символа или последовательности в поток
- `put`
 - записать в поток отдельный символ
- `write`
 - записать в поток последовательность

Библиотеки I/O

- C – `<cstdio>`
 - `printf`, `scanf`,
 - `fprintf`, `fscanf`
 - `sprintf`, `sscanf`
- C++ – `<iostream>`

Стандартные потоки C

макросы, разрешаются в `std::FILE*`

`stdin` – буферизуется

`stdout` – буферизуется

`stderr` – не буферизуется

Синхронизация С и С++ потоков

умолчательно потоки синхронизованы

отключение может использоваться для ускорения

```
std::ios::sync_with_stdio(false);
```

Логирование

(инструмент отладки)

- добавление в код отписывания сообщений
 - внутреннее состояние
 - принятые решения и переходы
- отключается в финальной версии

Логирование

(в больших проектах)

- использовать специальные библиотеки
 - удаленная отладка (локально и/или по сети)
 - с конфигурированием
 - с использованием уровней
 - с ротацией логов и т.д.
 - умолчательно выключено или минимально

ваши вопросы